Data Handling: Import, Cleaning and Visualisation

Sample Solution for Exercise/Workshop 3: Programming with Data/HTML Web Scraping

Aurélien Sallin (lecturer) and Federica Mascolo (class tutor)

24/10/2024

Exercises

Exercise A: Downloading and importing data on air quality

In the following, we are going to download and import data on air quality in New York.

1. Using a browser of your choice, navigate to this page by the US data.gov data catalog: https://catalog.data.gov/dataset/air-quality.
2. Scroll down to the Section 'Downloads & Resources' and download the data file in .csv format and put it into your `r_course/data` folder.
3. How do you open the data set? Try at least two alternatives. Hint: How did you do it in class?

The completed command reads

```
# install the readr package if not yet installed: install.packages("readr")
df_airquality <- read_delim("Air_Quality.csv", delim = ",", col_names = TRUE)
```

```
## Rows: 16122 Columns: 12
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (7): Name, Measure, Measure Info, Geo Type Name, Geo Place Name, Time Period, Start_Date
## dbl (4): Unique ID, Indicator ID, Geo Join ID, Data Value
## lgl (1): Message
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
df_airquality <- read.table("Air_Quality.csv", header = TRUE, sep = ",")
head(df_airquality, 5)
```

```
##   Unique.ID Indicator.ID                Name Measure Measure.Info Geo.Type.Name Geo.Join.ID
## 1    216498          386           Ozone (O3)    Mean          ppb            CD         313
## 2    216499          386           Ozone (O3)    Mean          ppb            CD         313
## 3    219969          386           Ozone (O3)    Mean          ppb       Borough           1
## 4    219970          386           Ozone (O3)    Mean          ppb       Borough           1
## 5    164876          383 Sulfur Dioxide (SO2)    Mean          ppb            CD         211
##                    Geo.Place.Name    Time.Period Start_Date Data.Value Message
## 1              Coney Island (CD13)    Summer 2013 06/01/2013      34.64      NA
## 2              Coney Island (CD13)    Summer 2014 06/01/2014      33.22      NA
## 3                           Bronx    Summer 2013 06/01/2013      31.25      NA
## 4                           Bronx    Summer 2014 06/01/2014      31.15      NA
## 5 Morris Park and Bronxdale (CD11) Winter 2008-09 12/01/2008       5.89      NA
```

Take a look at the data, do you notice differences?

4. Once you have loaded the data into RStudio, you can inspect the first (or last) couple of rows using `head(df_airquality)` (or `tail(df_airquality)`).

```
head(df_airquality)
```

```
##   Unique.ID Indicator.ID                 Name Measure Measure.Info Geo.Type.Name Geo.Join.ID
## 1    216498          386           Ozone (O3)    Mean          ppb            CD         313
## 2    216499          386           Ozone (O3)    Mean          ppb            CD         313
## 3    219969          386           Ozone (O3)    Mean          ppb       Borough           1
## 4    219970          386           Ozone (O3)    Mean          ppb       Borough           1
## 5    164876          383 Sulfur Dioxide (SO2)    Mean          ppb            CD         211
## 6    164877          383 Sulfur Dioxide (SO2)    Mean          ppb            CD         212
##                        Geo.Place.Name    Time.Period Start_Date Data.Value Message
## 1                  Coney Island (CD13)    Summer 2013 06/01/2013      34.64      NA
## 2                  Coney Island (CD13)    Summer 2014 06/01/2014      33.22      NA
## 3                               Bronx    Summer 2013 06/01/2013      31.25      NA
## 4                               Bronx    Summer 2014 06/01/2014      31.15      NA
## 5     Morris Park and Bronxdale (CD11) Winter 2008-09 12/01/2008       5.89      NA
## 6 Williamsbridge and Baychester (CD12) Winter 2008-09 12/01/2008       5.75      NA
```

5. Create a text file with the amount (in variable `Data.Value`) for fine particulate matter pollution (variable `Name == "Fine Particulate Matter (PM2.5)"`) in the winter of 2008/09 (variable `Time.Period == "Winter 2008-09"`) in lower Manhattan (variable `Geo.Place.Name == "Lower Manhattan"`). Save your text file as "airquality_manhattan.txt" in your `r_course/data` folder. Hint: use the `subset()` command, and note that you can link different subsetting conditions with the `&` operator.

Here, we first need to find the location of the information we are looking for. One option to do this is the built-in `subset()` function, using the `&` logical operator to connect the different conditions needed. The `==` operator is a comparison operator testing for logical equality, returning `TRUE` if the condition holds, and `FALSE` otherwise. Using just one `=` would produce an error because `R` expects a logical statement here.

```
my_info <- subset(df_airquality,
    Name == "Fine Particulate Matter (PM2.5)" &
    Geo.Place.Name == "Lower Manhattan" &
    Time.Period == "Winter 2008-09"
  )

my_value <- my_info$Data.Value

cat(my_value, file = "airquality_manhattan.txt") # concat and save
```

The resulting file should contain the value `15.79`

Exercise B: Read raw text data

This exercise guides you through how we can import raw text data into R. Suppose you are writing your BA Thesis on the topic "The Effect of Climate Change on Consumption Behavior in Switzerland". For your empirical analysis you need, inter alia, standardized data on temperature anomalies in Switzerland (as a proxy for the salience of climate change). You identify NASA's Earth System Data Explorer as a promising data source and aim to download and work with this data in R. The exercise guide's you exemplary though how this can be done.

Hint: This exercise is an adapted version of the tutorial in Chapter 1 of Paul Murell's book (see syllabus). Read the chapter for an additional introduction to data stored in text files.

1. Go to NASA's Earth System Data Explorer: https://mynasadata.larc.nasa.gov/EarthSystemLAS/UI.vm.
2. Click on *Data Set Selection* in the upper left corner and select Atmosphere (in 1. Select a Sphere of the hearth system), select Temperature (in 2.), select Monthly Surface Air Temperature (in 3.) and select the most recent date (in 4.)
3. On the RHS select `Time Plot` and enter coordinates 47.25 N and 9.22 W (St. Gallen) and click `Generate Plot`. The page shows a plot of the monthly surface air temperature anomaly time series.
4. Click on `Generate Data Table` and save is as .csv.

Since we already know how to read a csv, I will use the old data for this application and load them as .txt, You can find the old data on Canvas.

```
# install the readr package if not yet installed: install.packages("readr")
library(readr)
# read the data
ta <- read_tsv("temp_anomaly.txt", skip = 9, col_names = c("month", "temp_anomaly"))
```

```
## Rows: 1713 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (1): month
## dbl (1): temp_anomaly
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# inspect the first rows
head(ta)
```

```
## # A tibble: 6 x 2
##   month    temp_anomaly
##   <chr>           <dbl>
## 1 JAN-1880         0.33
## 2 FEB-1880         0.3
## 3 MAR-1880         0.47
## 4 APR-1880         0.15
## 5 MAY-1880        -0.36
## 6 JUN-1880        -0.43
```

6. What is the purpose of the `skip` argument and why is it set to `9`?

In the raw text file, we see that the first 9 lines contains metadata and are in a different format - therefore we need to skip these lines when reading the data into R. Also note how we specified the column names in our command to read in the data, because the data did not have the variable names included in the columns.

7. Why do we use `read_tsv()` in this situation and not `read_csv()`?

The raw data shows us that columns are not separated by a comma, but rather by a tab, hence we have what is called a tab-separated values (TSV) file. That said, we could also use the more general ***`read_delim()`*** *function, specifying the tab* `\t` *as a separator:*

```
ta <- read_delim("temp_anomaly.txt", skip = 9, delim = "\t",
                 col_names = c("month", "temp_anomaly"))
```

```
## Rows: 1713 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
```

```
## chr (1): month
## dbl (1): temp_anomaly
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Exercise C: Importing data on air quality (XML format)

1. I provided you with a data file `Air_Quality.xml` on Canvas. Save it in your `r_course/data` folder.
2. Load the data in .xml format. Hint: The package `xml2` might be of use. You can install new packages as follows: `install.packages("package_name")`.

```r
# Installing the package if not already installed
if (!require(xml2)) {
  install.packages("xml2")
}
```

```
## Loading required package: xml2
```

```
## Warning: package 'xml2' was built under R version 4.3.3
```

```r
library(xml2) # Loading the package

# Load the data from the XML file
xml_data <- read_xml("Air_Quality.xml")


# understand the structure before transforming to a data frame
#xml_structure(xml_data)
#xml_text(xml_data)


# Define the variable names and corresponding XPath expressions
variables <- c("unique_id" = "//unique_id",
               "indicator_id" = "//indicator_id",
               "name" = "//name",
               "measure" = "//measure",
               "measure_info" = "//measure_info",
               "geo_type_name" = "//geo_type_name",
               "geo_join_id" = "//geo_join_id",
               "geo_place_name"  = "//geo_place_name",
               "time_period"  = "//time_period",
               "start_date" = "//start_date",
               "data_value" = "//data_value" )

# Use lapply to extract the text for each variable and create a tibble
air_quality_df <- tibble::as_tibble(lapply(variables, function(xpath) xml_text(xml_find_all(xml_data, xp

# Set the column names of the tibble
names(air_quality_df) <- names(variables)

# Print the resulting tibble
air_quality_df
```

```
## # A tibble: 16,122 x 11
##    unique_id indicator_id name         measure measure_info geo_type_name geo_join_id geo_place_name
```

```
##    <chr>     <chr>        <chr>        <chr>   <chr>        <chr>         <chr>       <chr>
##  1 216498    386          Ozone (O3)   Mean    ppb          CD            313         Coney Island ~
##  2 216499    386          Ozone (O3)   Mean    ppb          CD            313         Coney Island ~
##  3 219969    386          Ozone (O3)   Mean    ppb          Borough       1           Bronx
##  4 219970    386          Ozone (O3)   Mean    ppb          Borough       1           Bronx
##  5 164876    383          Sulfur Diox~ Mean    ppb          CD            211         Morris Park a~
##  6 164877    383          Sulfur Diox~ Mean    ppb          CD            212         Williamsbridg~
##  7 219971    386          Ozone (O3)   Mean    ppb          Borough       2           Brooklyn
##  8 219972    386          Ozone (O3)   Mean    ppb          Borough       2           Brooklyn
##  9 164878    383          Sulfur Diox~ Mean    ppb          CD            301         Greenpoint an~
## 10 164879    383          Sulfur Diox~ Mean    ppb          CD            302         Fort Greene a~
## # i 16,112 more rows
## # i 3 more variables: time_period <chr>, start_date <chr>, data_value <chr>
```

```r
# check the structure
str(air_quality_df) # Checking structure again
```

```
## tibble [16,122 x 11] (S3: tbl_df/tbl/data.frame)
##  $ unique_id     : chr [1:16122] "216498" "216499" "219969" "219970" ...
##  $ indicator_id  : chr [1:16122] "386" "386" "386" "386" ...
##  $ name          : chr [1:16122] "Ozone (O3)" "Ozone (O3)" "Ozone (O3)" "Ozone (O3)" ...
##  $ measure       : chr [1:16122] "Mean" "Mean" "Mean" "Mean" ...
##  $ measure_info  : chr [1:16122] "ppb" "ppb" "ppb" "ppb" ...
##  $ geo_type_name : chr [1:16122] "CD" "CD" "Borough" "Borough" ...
##  $ geo_join_id   : chr [1:16122] "313" "313" "1" "1" ...
##  $ geo_place_name: chr [1:16122] "Coney Island (CD13)" "Coney Island (CD13)" "Bronx" "Bronx" ...
##  $ time_period   : chr [1:16122] "Summer 2013" "Summer 2014" "Summer 2013" "Summer 2014" ...
##  $ start_date    : chr [1:16122] "2013-06-01T00:00:00" "2014-06-01T00:00:00" "2013-06-01T00:00:00" "
##  $ data_value    : chr [1:16122] "34.64" "33.22" "31.25" "31.15" ...
```

```r
# Alternative with XML package:
if (!require(XML)) {
install.packages("XML")
}
```

```
## Loading required package: XML
```

```
## Warning: package 'XML' was built under R version 4.3.3
```

```r
library(XML) # loading the package after install - need to run every time we use it!
# Load the data from the XML file
xml_data <- xmlParse("Air_Quality.xml")
# Convert XML data to a data frame
air_quality_data <- xmlToDataFrame(xml_data)

# check the structure
str(air_quality_data) # Checking structure again
```

```
## 'data.frame':    16122 obs. of  11 variables:
##  $ unique_id     : chr  "216498" "216499" "219969" "219970" ...
##  $ indicator_id  : chr  "386" "386" "386" "386" ...
##  $ name          : chr  "Ozone (O3)" "Ozone (O3)" "Ozone (O3)" "Ozone (O3)" ...
##  $ measure       : chr  "Mean" "Mean" "Mean" "Mean" ...
##  $ measure_info  : chr  "ppb" "ppb" "ppb" "ppb" ...
##  $ geo_type_name : chr  "CD" "CD" "Borough" "Borough" ...
##  $ geo_join_id   : chr  "313" "313" "1" "1" ...
##  $ geo_place_name: chr  "Coney Island (CD13)" "Coney Island (CD13)" "Bronx" "Bronx" ...
```

```
##  $ time_period   : chr  "Summer 2013" "Summer 2014" "Summer 2013" "Summer 2014" ...
##  $ start_date    : chr  "2013-06-01T00:00:00" "2014-06-01T00:00:00" "2013-06-01T00:00:00" "2014-06-0
##  $ data_value    : chr  "34.64" "33.22" "31.25" "31.15" ...
```

We can see that we loaded an object, but instead of being a nice neat dataframe, what we got was actually an R structure representing the XML tree, i.e. a structure that contains all the information we need to extract the data we want, but we cannot use this directly like a data frame

We have a data frame now - with the caveat that the reading function defaulted to declaring all variables as strings. In order to be able to work with numerical columns, we would first have to convert them into numeric variables, which can be done like this for example: `air_quality_data$data_value <- as.numeric(air_quality_data$data_value)`*. For our purposes, importing data from a .csv or text file is often easier than parsing XML files (note also the file size of the XML data relative to the csv file, in itself not the most compact format either).*

Exercise D: Exploring computer code - HTML - let's manipulate the media!

In this exercise, we will explore HTML code, which is the basis of most websites.

1. Open your preferred web browser (e.g. Firefox, Chrome or Safari. Note that Internet Explorer is never a preferred browser). Go to the following website: `https://www.foxnews.com/`.
2. Now let's look at the HTML source code for this website - see *here* for information on how to do this in your browser of choice. Try to recognize parts of the code that correspond to elements of the displayed page.
3. Pick a news headline on the site and find it in the source code - on Windows, you can use `CTRL + F` to use the search bar to look for it in the source code by copying the first couple of words from the headline and putting it into the search bar. Note: most browsers also have an 'inspect element' option when you right click on/select the element you want to look at. This can take you directly to the corresponding part of the HTML (or at least to the vicinity, sometimes you also have to try a couple of times to get where you want to be). For more information on how to do this (in Safari, you will need to activate developer mode for example), see the following *link*

For example, I found the following segment in the HTML code relating to an article titled 'Republicans sound alarm over CCP-linked businessman's cash offer to Hunter Biden' (no subtle political messaging intended):

```
<!DOCTYPE html>
<html>

<head>
...

</head>

<body>

...

  <article class="article story-2">
      <div class="m">
          <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy-
relationship-ccp-linked-businessman-damning-evidence" data-omtr-intcmp="fnhpbt3" data-adv=
  "hp1bt">

            <picture>
                <source media="(max-width: 767px)" srcset="//a57.foxnews.com/prod-hp.fox
news.com/images/2023/10/343/193/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1, //a57.
```

```
foxnews.com/prod-hp.foxnews.com/images/2023/10/686/386/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1 2x">
                <source media="(min-width: 768px) and (max-width: 1023px)" srcset="//a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/335/188/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1, //a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/670/376/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1 2x">
                <source media="(min-width: 1024px) and (max-width: 1279px)" srcset="//a57.
                foxnews.com/prod-hp.fox
news.com/images/2023/10/303/170/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1, //a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/606/340/57
6c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1 2x">
                <source media="(min-width: 1280px) and (max-width: 1439px)" srcset="//a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/277/156/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1, //a57.foxnews.com/prod-hp.foxnews.com/images/2023/10/554/312/576c017a6f
05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1 2x">
                <source media="(min-width: 1440px)" srcset="//a57.foxnews.com/prod-hp.fox
news.com/images/2023/10/331/186/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1, //a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/662/372/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1 2x">
                <img src="//a57.foxnews.com/prod-hp.foxnews.com/images/2023/10/343/193/576
c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1" width="343" height="193" alt="Republicans
sound alarm over CCP-linked businessman's cash offer to Hunter Biden">
            </picture>
            <div class="kicker default"><span class="kicker-text">'IN THEIR POCKET'</span>
            </div>
          </a>
          <span class="pill is-breaking">EXCLUSIVE</span>
      </div>

      <div class="info ">
        <header class="info-header">
          <h3 class="title ">
            <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy
            -relationship-ccp-linked-businessman-damning-evidence#&amp;_intcmp=fnhpbt3"
            data-omtr-intcmp="fnhpbt3">Republicans sound alarm over CCP-linked business-
            man's cash offer to Hunter Biden</a>
          </h3>
        </header>
        <div class="content">


        </div>
      </div>

  </article>

...

</body>

</html>
```

4. Now it's time for some good old media manipulation. In the HTML-code, replace the chosen news

headline with a title of your choice (if you need some inspiration, see *here*). In most browsers, you can double click the source code to edit it, then confirm your edits by hitting the enter/return key. What can you see? Did the website change?

*For example, we could change the displayed story title by changing the text of the

element:*

```
<!DOCTYPE html>
<html>

<head>
...

</head>

<body>

  ...

      <div class="info ">
        <header class="info-header">
          <h3 class="title ">
            <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy
            -relationship-ccp-linked-businessman-damning-evidence#&amp;_intcmp=fnhpbt3"
            data-omtr-intcmp="fnhpbt3">In surprise move, Republicans turn to full support
            for Joe Biden. Country now in peace and harmony.</a>
          </h3>
        </header>
      </div>

  </article>

...

</body>

</html>
```

5. BONUS (not exam relevant): Can you figure out a way to replace the article teaser image as well? Hint: this is easiest if you use online image addresses, e.g. https://static.wikia.nocookie.net/mrbean/images/4/4b/Mr_beans_holiday_ver2.jpg or https://preview.redd.it/bad-luck-brian-meme-hd-upscaled-v0-syiviojoq72a1.jpg?width=1080&crop=smart&auto=webp&s=eace4bc6be838bfe3f30219743d5a5c34c5f4335)

The simplest way of doing this is to replace the link to the image with a link to another image in the element (note that I also removed all the sub-elements starting with `<source media=?`*, these are just what the webpage authors implemented to allow for choosing different image files depending on the screen size - you would have to replace each image link to ensure full replacement in case you were to keep those bit in)*

```
<!DOCTYPE html>
<html>


<head>
...
```

```
</head>

<body>

...

  <picture>
      <img src="https://cdn.pixabay.com/photo/2016/05/10/21/50/meditation-1384758_1280.jpg"
      width="343" height="193" alt="In surprise move, Republicans turn to full support
      for Joe Biden. Country now in peace and harmony.">
  </picture>

...

</body>

</html>
```

As a sidenote, the `alt="In surprise move, Republicans turn to full support for Joe Biden. Country now in peace and harmony."` *part is not the same as the title we changed earlier, it defines the alternative text that describes the image, which serves purposes like assisting accessibility and search engine optimization (SEO).*

Exercise E: Web Scraping Exercise (advanced): Importing Data from a HTML Table (on a website)

Previously, we discussed the *Hypertext Markup Language (HTML)* as code to define the structure/content of a website and HTML-documents as semi-structured data sources. The following exercise contains the basic steps necessary to import data from an HTML table into R.

The aim of the exercise is to generate a CSV file containing data on 'divided government' in US politics. We use the following Wikipedia page as a data source: https://en.wikipedia.org/wiki/Divided_government_in_the_United_States. The page contains a table indicating the president's party, and the majority party in the US House and the US Senate per Congress (2-year periods). The first few rows of the cleaned data are supposed to look like this:

```
##     year  president senate  house
##    <int>     <char> <char> <char>
## 1:  1861    Lincoln      R      R
## 2:  1862    Lincoln      R      R
## 3:  1863    Lincoln      R      R
## 4:  1864    Lincoln      R      R
## 5:  1865 A. Johnson      R      R
## 6:  1866 A. Johnson      R      R
```

In a first step, we initiate fix variables for paths and load additional R packages needed to handle data stored in HTML documents. NOTE: the `OUTPUT_PATH` specified here assumed that there is a folder called `data` in the current working directory (get your current working directory by running `getwd()`) - adjust it if you that is not the case or if you want to store the output somewhere else.

```
# SETUP ------------------

# load packages
library(rvest)
```

```
## Warning: package 'rvest' was built under R version 4.3.3
```

```
##
## Attaching package: 'rvest'

## The following object is masked from 'package:readr':
##
##     guess_encoding
```

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.3.3

## Warning: package 'ggplot2' was built under R version 4.3.3

## Warning: package 'tidyr' was built under R version 4.3.3

## Warning: package 'forcats' was built under R version 4.3.3

## -- Attaching core tidyverse packages ------------------------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v purrr     1.0.2
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1

## -- Conflicts ------------------------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter()         masks stats::filter()
## x rvest::guess_encoding() masks readr::guess_encoding()
## x dplyr::lag()            masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
# fix vars
SOURCE_PATH <- "https://en.wikipedia.org/wiki/Divided_government_in_the_United_States"
OUTPUT_PATH <- "data/divided_gov.csv"
```

Now we write the part of the script that fetches the data from the Web. This part consists of three steps.

1. Fetch the entire website (HTML document) from Wikipedia with (`read_html()`).

```
# FETCH/FORMAT DATA --------------------

# fetch from web
doc <- read_html(SOURCE_PATH)
```

2. Extract the part of the website containing the table with the data we want (hint: use `html_table()` on the HTML document you just fetched, and see if one of the resulting tables in the list generated by this function matches what we are looking for, you also want the option `fill=TRUE` inside that function).

```
tab <- html_table(doc)[[2]] # select the second table from the list we get
```

3. Finally, check that the table has been properly parsed (you may want to remove the last line if it contains code junk) and store its content in a data frame called `tab`.

When inspecting the table, we see that the last row also contains the column names - will remove this (note that the minus sign can be used for a negative selection, i.e. to select rows that you do not want, and ***nrow*** *will give us the index of the last, n-th row):*

```
tab <- tab[-nrow(tab), ] # remove last row (not containing data)
```

4. Clean the data to get a data set more suitable for data analysis. Note that the original table contains information per congress (2-year periods). However, as the sample above shows, we aim for a panel at the year level, i.e. there should be a separate observation for every individual year. The following code iterates through the rows of the data frame and generates for each row per congress several two rows

(one for each year in the congress).[1] Try to understand what each line of the following code does (it should run as is if the data frame `tab` had the right format):

Now things get a bit more complicated - see my comments in the code:

```
all_years <- list() # creating empty list object to
n <- length(tab$Year) # determining the number of years we have to iterate over
length(all_years) <- n # pre-allocating space in the list (not strictly necessary,
# but can improve speed/memory efficiency in some cases)

for (i in 1:n){

      row <- tab[i,] # choosing row number i of the input table
      y <- row$Year  # extracting current year

      # In the following line, we extract the beginning year,
      # for example if the year column says '2005-2007', begin would take the value '2005'
      # and end would take the value '2007'.
      # This is done in three steps:
      #   - first, strsplit() creates a list with two elements based on a regular
      #     expression "[\\-\\-]", which  indicates that the split should be before
      #     and after either a '-' or '-' character. The option 'perl = TRUE' enables
      #     use of Perl-compatible (read: more intricate/fancy) regular expressions.
      #     To understand the elements of a regular expression, ChatGPT or tools such as
      #     'regexr.com' can be useful
      #   - second, the 'unlist()' command converts the list of elements generated by
      #     strsplit() into a vector, and we choose the first element for the 'begin'
      #     part, and the second element (end year) for the 'end' part
      #   - third, we convert the year character string into a numeric value with
      #     'as.numeric()'
      begin <- as.numeric(unlist(strsplit(x = y, split = "[\\-\\-]", perl = TRUE))[1])
      end <- as.numeric(unlist(strsplit(x = y, split =  "[\\-\\-]", perl = TRUE))[2])

      # with these two values, we then create a new data frame where we have all the
      # years from begin to end (not including end) in the first column, and the parties
      # of president, senate and house majority in columns 2-4. Note that we exclude the
      # end year because congress is handed over in January of the final year, meaning the
      # final year is actually already the year the next congress is in power.
      # In addition, note that r will recycle the single value provided in columns 2-4
      # as often as needed to fill the number of rows of the data frame. R does this as
      # long as the longest column vector is an exact multiple of the shorter vectors -
      # KEEP IN MIND, AS THIS CAN LEAD TO UNDESIRED BEHAVIOR IF YOU DO NOT WANT THIS TO
      # HAPPEN
      tabnew <- data.frame(year=begin:(end-1), president=row$President, senate=row$Senate,
                           house=row$House)

      all_years[[i]] <- tabnew # adding the newly create observations to the list
}

allyears <- bind_rows(all_years) # combining our list of observations into a data frame
```

5. In a last step, inspect the collected data and write it to a CSV file, using the `OUTPUT_PATH` defined at the beginning of this exercise to specify where the file should be stored.

[1] See `?strsplit`, `?unlist`, and this introduction to regular expressions for the background of how this is done in the code example here.

```r
# WRITE TO DISK -----------------------------------------------

# inspect
head(allyears)
```

```
##   year  president senate house
## 1 1861    Lincoln      R     R
## 2 1862    Lincoln      R     R
## 3 1863    Lincoln      R     R
## 4 1864    Lincoln      R     R
## 5 1865 A. Johnson      R     R
## 6 1866 A. Johnson      R     R
```

```r
# note: with head, you may not always get the best impression, and with a small dataframe
# such as this one, viewing the entire one may be a better option to spot any pot. issues:
view(allyears)

# remove unwanted Wikipedia footnote references '[a]' etc., note again the use of a regular
# expression to do so
allyears$senate <- str_remove_all(allyears$senate, "\\[[^\\]]*\\]")
allyears$house <- str_remove_all(allyears$house, "\\[[^\\]]*\\]")
#  remove [] -- [^\\]]* means "match zero or more characters that are not a closing

# write to CSV
write_csv(allyears, file=OUTPUT_PATH)
```